

Clean software development

Day 3

Alessandro Sciarra

Z02 – Software Development Center

8 October 2025

Topics of the day

1 Git in real life

- Git tag
- Semantic versioning
- Git-flow
- Summary and outlook

2 Clean testing (IV)

3 TDD live demo

4 Practice TDD (I)

5 Practice TDD (II)

6 GitHub PR & co.

7 Sum up and closing

An exciting day

Today you will broaden your boundaries and hopefully never come back to the old habits!

Git in real life

Git tag

Tagging the history of a repository

Lightweight tags

They simply are a name for an object (often a commit) and they are usually meant for private or temporary use

```
$ git tag v1.0
$ git tag
v1.0
$ git show v1.0
commit aa879f463acd41fc38c7e96090cc1eea279304df
Author: Alessandro Sciarra <sciarra@itp.uni-frankfurt.de>
Date:   Wed Aug 16 14:01:08 2025 +0200
```

```
Coffee machine production ready
```

```
# Commit differences
```



Tagging the history of a repository

Lightweight tags

They simply are a name for an object (often a commit) and they are usually meant for private or temporary use

Annotated tags

Tag objects contain a creation date, the tagger name and e-mail, a tagging message, and an optional GnuPG signature



Tagging the history of a repository

```
$ git tag -a v1.0 -m\  
> "Coffee machine software ready for production"  
$ git show v1.0  
tag v1.0  
Tagger: Alessandro Sciarra <sciarra@itp.uni-frankfurt.de>  
Date:   Wed Aug 17 09:12:24 2025 +0200  
  
Coffee machine software ready for production  
  
commit aa879f463acd41fc38c7e96090cc1eea279304df  
Author: Alessandro Sciarra <sciarra@itp.uni-frankfurt.de>  
Date:   Wed Aug 16 14:01:08 2025 +0200  
  
Coffee machine production ready  
  
# Commit differences
```

Prefer annotated tags for releases!



Tagging the history of a repository

Lightweight tags

They simply are a name for an object (often a commit) and they are usually meant for private or temporary use

Annotated tags

Tag objects contain a creation date, the tagger name and e-mail, a tagging message, and an optional GnuPG signature

Tags as commits are by default local

Use `git push --tags` to push them



Tagging a code: Let's speak about the **same** code!

A milestone in development

A tag in the git history is to some extent a commitment, but it is also a statement of invaluable help about the codebase!

Why should I?

- The codebase is released
- The codebase will be released
- The codebase is private but shared with colleagues
- The codebase will be (maybe) inherited
- **The software is used to produce data**



Semantic versioning

How should tags be named?



How should tags be named?

[Prefix]X[.Y[.Z]]

According to the  Semantic Versioning, increase

MAJOR version **X** when you make incompatible API changes

MINOR version **Y** when you add functionality in a backwards-compatible manner

PATCH version **Z** when you make backwards-compatible bug fixes



How should tags be named?

[Prefix]X[.Y[.Z]]

According to the  Semantic Versioning, increase

MAJOR version **X** when you make incompatible API changes

MINOR version **Y** when you add functionality in a backwards-compatible manner

PATCH version **Z** when you make backwards-compatible bug fixes

Choose your alternative, but give yourself a rule, e.g. increase

MAJOR version **X** when you introduce new big features

MINOR version **Y** when you add minor functionality or do big refactoring

PATCH version **Z** when you make bug fixes (without large changes)



How should tags be named?

[Prefix]X[.Y[.Z]]

According to the  Semantic Versioning, increase

MAJOR version **X** when you make incompatible API changes

MINOR version **Y** when you add functionality in a backwards-compatible manner

PATCH version **Z** when you make backwards-compatible bug fixes

Choose your alternative, but give yourself a rule, e.g. increase

MAJOR version **X** when you introduce new big features

MINOR version **Y** when you add minor functionality or do big refactoring

PATCH version **Z** when you make bug fixes (without large changes)

MAJOR version **X** when you introduce new features

MINOR version **Y** when you do big refactoring or fix bugs



How should tags be named?

[Prefix]X[.Y[.Z]]

According to the  Semantic Versioning, increase

MAJOR version **X** when you make incompatible API changes

MINOR version **Y** when you add functionality in a backwards-compatible manner

PATCH version **Z** when you make backwards-compatible bug fixes

Choose your alternative, but give yourself a rule, e.g. increase

MAJOR version **X** when you introduce new big features

MINOR version **Y** when you add minor functionality or do big refactoring

PATCH version **Z** when you make bug fixes (without large changes)

MAJOR version **X** when you introduce new features

MINOR version **Y** when you do big refactoring or fix bugs

However you do it, use a CHANGELOG file to list user-relevant changes!



Ideally

```
$ git tag -n
v2.0.0    Coffee machine ready for milk drinks    Major
v1.3.1    Fix milk temperature problem           Patch
v1.3.0    Interface milk system with machine      Minor
v1.2.0    Add foam generator                     Minor
v1.1.0    Add milk container                     Minor
v1.0.0    Coffee machine production ready        Major
v0.7.1    Fix pipe failures                     Patch
v0.7.0    Interface all components               Minor
v0.6.0    Add coffee grounds container           Minor
v0.5.0    Add water tray                        Minor
v0.4.0    Add water tank                        Minor
v0.3.0    Add bean container                    Minor
v0.2.0    Add brew system and core engine        Minor
v0.1.0    Deploy coffee machine skeleton         Minor
```

- Clear evolution of the codebase for everybody
- **v1.0** usually refers to the first **production-ready** version



Git-flow

3

Git in real life

- Git tag
- Semantic versioning
- **Git-flow**
 - Motivation and background
 - The main branches
 - Feature branches
 - Release branches
 - Hotfix branches
 - Git-flow: an additional tool
- Summary and outlook



Disclaimer and references

Vincent Driessen back in 2010

In the following I will give you a summary of his blog page.
All credit goes to the author and no original part is here contained.

Vincent Driessen as nvie

The original author also implemented a collection of git extensions which allow to easily use his branching model (active till 2012).

Alternative implementations

AVH Edition:  gitflow-avh (archived in June 2023)

CJS Edition:  gitflow-cjs (still active, but rarely)



Motivation



	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

<https://www.xkcd.com/1296/>



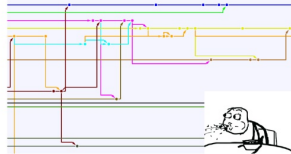
Motivation

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFTSLKDFJSOKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

<https://www.xkcd.com/1296/>

Sometimes history becomes messy



[https://image.slidesharecdn.com/\[...\]](https://image.slidesharecdn.com/[...])

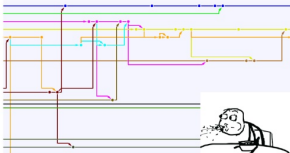
Motivation

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

<https://www.xkcd.com/1296/>

Sometimes history becomes messy

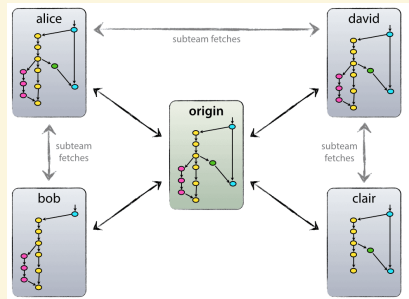


[https://image.slidesharecdn.com/\[...\]](https://image.slidesharecdn.com/[...])

- **Ordered** and **standardized** way to daily work
- Easy way to keep big projects history and development tidied up
- Sustainable work in team also **for new members**
- In larger projects, easy interaction within and between sub-teams
- A way to go for released software

Background

- Central repository (e.g. GitHub)
- Developers pull/push from/to origin
- Sub-team fetches lead to high work efficiency
- It might be useful to work together with two or more developers on a big new feature, before pushing the work in progress to origin prematurely.
- **Never forget origin is public!**



Technically, this means nothing more than that, for example, Alice has defined a Git remote, named `bob`, pointing to Bob's repository, and vice versa.

The main and develop branches

`origin/main`

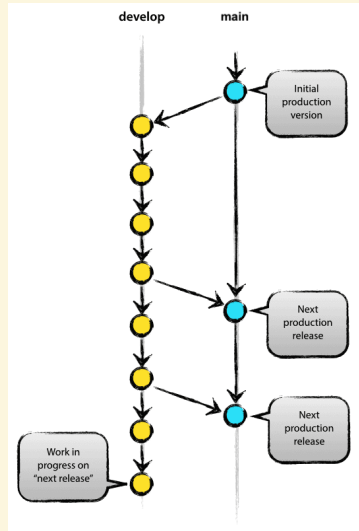
The source code of HEAD always reflects a production-ready state

`origin/develop`

The source code of HEAD always reflects a state with the latest delivered development changes for the next release

Each time when changes are merged back into main, this is a new production release by definition

→ hooks for enforced policies!

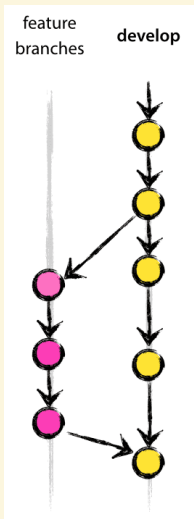


The feature branches

Properties

- May branch off from:
`develop`
- Must merge back into:
`develop`
- Any name except:
`main`, `develop`
`release-*`, `hotfix-*`

Feature branches typically exist in developer repository only, not in origin.

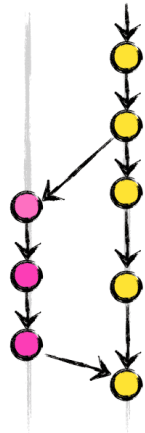


The feature branches

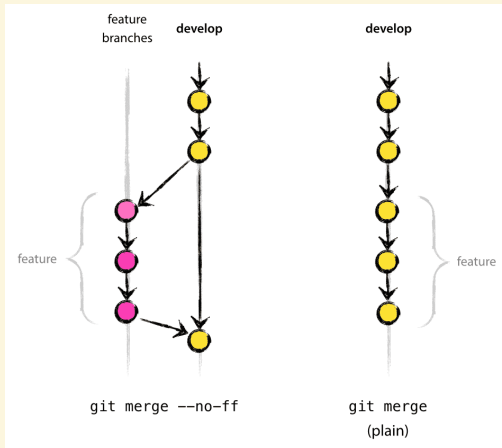
How it looks like:

```
$ git switch -c myfeature develop
Switched to a new branch "myfeature"
#-----
# Some development, commits, etc.
#-----
$ git switch develop
Switched to branch 'develop'
$ git pull origin develop
Already up-to-date.
$ git merge --no-ff myfeature
Updating ea1b82a..05e9557
(Summary of changes)
$ git branch -d myfeature
Deleted branch myfeature (was 05e9557).
$ git push origin develop
[...]
```

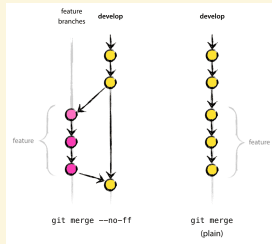
feature branches develop



Why to avoid a fast-forward merge?



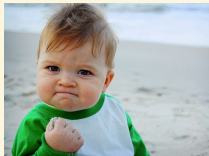
Why to avoid a fast-forward merge?



- ✓ No information lost about the historical existence of a feature
- ✓ Easy to see in history which commits have implemented a feature
- ✓ Easy to revert a whole feature (i.e. a group of commits)
- ✗ It will create (empty) commit objects
 - ↳ the gain is much bigger than the cost!

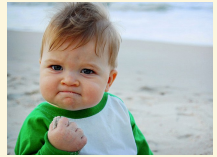
The release branches

Make you feel like



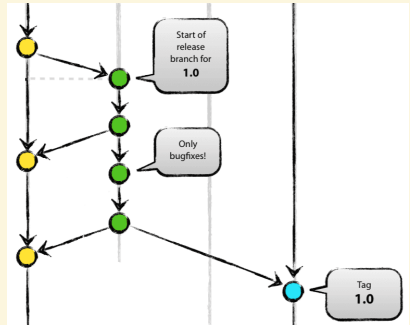
The release branches

Make you feel like



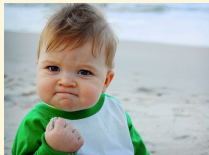
Preproperties:

- May branch off from:
develop
- Must merge back into:
develop, main
- Branch naming convention:
release-*



The release branches

Make you feel like



- Release branches support preparation of a new production release
- Last-minute checks, minor bug fixes and preparing meta-data (version number, etc.) should be done on a release branch
- By doing all of this work on a release branch, the **develop** branch is cleared to receive features for the following release
- When a new release branch is created, all features that are targeted for this release must be merged into the **develop** branch

The release branch: How it works

Creation

```
Version 1.1.5 is the current production release
We are ready for "next release" -> NOW we decide v1.2
$ git switch -c release-1.2 develop
Switched to a new branch "release-1.2"
$ ./bump-version.sh 1.2 #Just modify some files
Files modified successfully, version bumped to 1.2.
$ git commit -a -m "Bump version number to 1.2"
[release-1.2 74d9424] Bumped version number to 1.2
1 files changed, 1 insertions(+), 1 deletions(-)
```

Adding new features here is strictly prohibited!
The idea is to just make sure everything works and is
ready to be published!



The release branch: How it works

Creation

```
$ git switch -c release-1.2 develop
$ ./bump-version.sh 1.2 #Just modify some files
$ git commit -a -m "Bump version number to 1.2"
```

Finalisation

```
$ git switch main
Switched to branch 'main'
$ git merge --no-ff release-1.2
Merge made by recursive. (Summary of changes)
$ git tag -a v1.2
$ git switch develop
Switched to branch 'develop'
$ git merge --no-ff release-1.2 #Maybe conflicts!
Merge made by recursive. (Summary of changes)
$ git branch -d release-1.2
Deleted branch release-1.2 (was ff452fe).
```

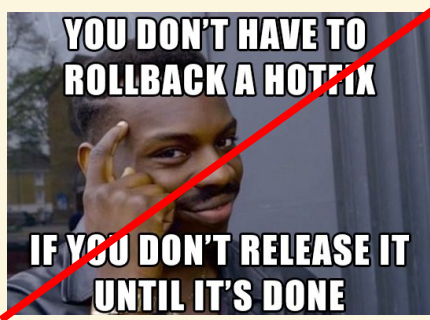


The hotfix branch



The hotfix branch

A hotfix is not something bad!



...and, after all, never forget Murphy's law!

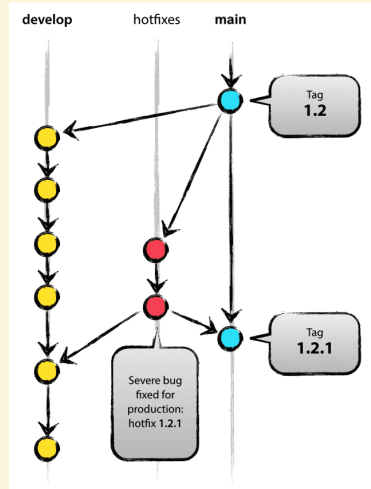
The hotfix branch

Properties:

- May branch off from:
main
- Must merge back into:
develop, main
- Branch naming convention:
hotfix-*

Main advantage

The essence is that work on the develop branch can continue, while a quick production fix is prepared



The hotfix branch: How it works

Creation

```
$ git switch -c hotfix-1.2.1 main
Switched to a new branch "hotfix-1.2.1"
$ ./bump-version.sh 1.2.1 #Just modify some files
Files modified successfully, version bumped to 1.2.1.
$ git commit -a -m "Bump version number to 1.2.1"
[hotfix-1.2.1 41e61bb] Bump version number to 1.2.1
1 files changed, 1 insertions(+), 1 deletions(-)
# Some work to fix the situation
$ git commit -m "Fix severe production problem"
[hotfix-1.2.1 abbe5d6] Fix severe production problem
5 files changed, 32 insertions(+), 17 deletions(-)
```



The hotfix branch: How it works

Creation

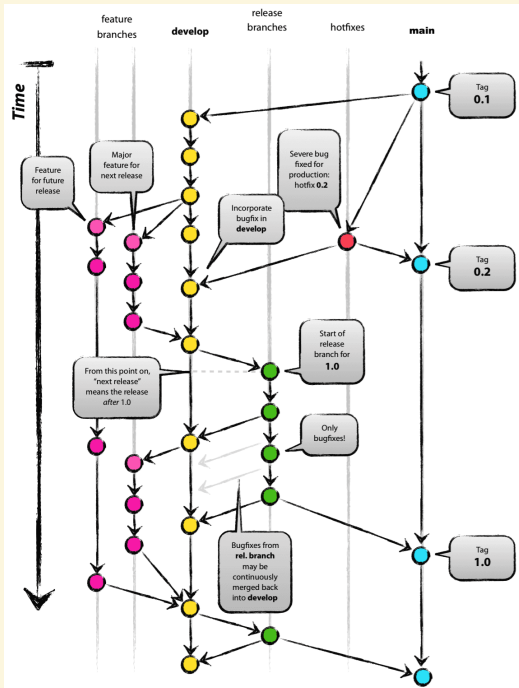
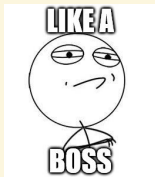
```
$ git switch -c hotfix-1.2.1 main
$ ./bump-version.sh 1.2.1 #Just modify some files
$ git commit -a -m "Bump version number to 1.2.1"
# Some work to fix the situation
$ git commit -m "Fix severe production problem"
```

Finalisation

```
$ git switch main
Switched to branch 'main'
$ git merge --no-ff hotfix-1.2.1
Merge made by recursive. (Summary of changes)
$ git tag -a v1.2.1
$ git switch develop
Switched to branch 'develop'
$ git merge --no-ff hotfix-1.2.1
Merge made by recursive. (Summary of changes)
$ git branch -d hotfix-1.2.1
Deleted branch hotfix-1.2.1 (was abbe5d6).
```



Summary





But, wait, everything by hand?!

Git-flow: How to easily apply what we learnt

Pick a set of high-level repository operations



nvie

or better



gitflow-avh



gitflow-cjs



Git-flow: How to easily apply what we learnt

Pick a set of high-level repository operations



or better



gitflow-avh



gitflow-cjs

The base syntax

→ Standard raw git commands can be used as usual, too

```
# To initialize a new repository, use:
git flow init [-d]
# To list/start/finish auxiliary branches, use:
git flow <type>
git flow <type> start <name> [<base>]
git flow <type> finish <name>
# To push/pull an auxiliary branch to the remote, use:
git flow <type> publish <name>
git flow <type> pull <remote> <name>
```

<type> can be either feature, release or hotfix

<name> is the name of the auxiliary branch

<base> must be a commit on develop or main



Git-flow: How to easily apply what we learnt

Pick a set of high-level repository operations



or better



gitflow-avh



gitflow-cjs

Take-home message

If you have a released software or plan to have one,
Git-flow branching model can help you to bring your
product to another level!

Think about it

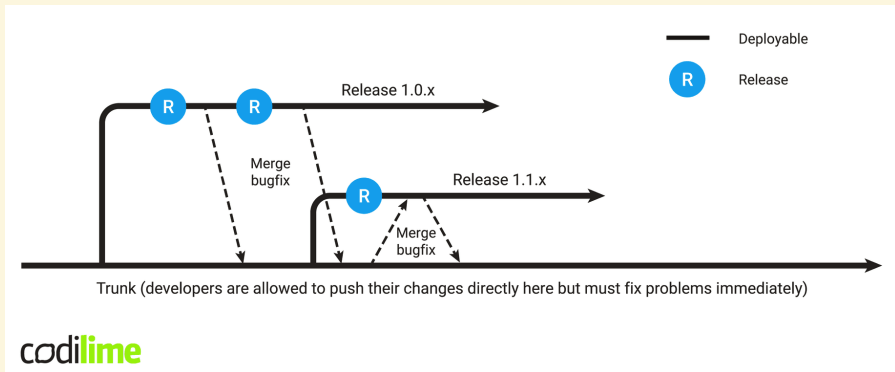
Even if you use a (slightly) different branching model, it might well be
that available git extensions still partially fit your use case



I heard Git-flow is obsolete, is it?

No, it is not!

Trunk-based development



In the last years this alternative approach has become popular



No, it is not!

When does trunk-based development work best?

- When you need to iterate quickly
- When you work mostly with senior developers

When does Git-flow work best?

- When you have a lot of junior developers
- When you have an established product
- When you run an open-source project

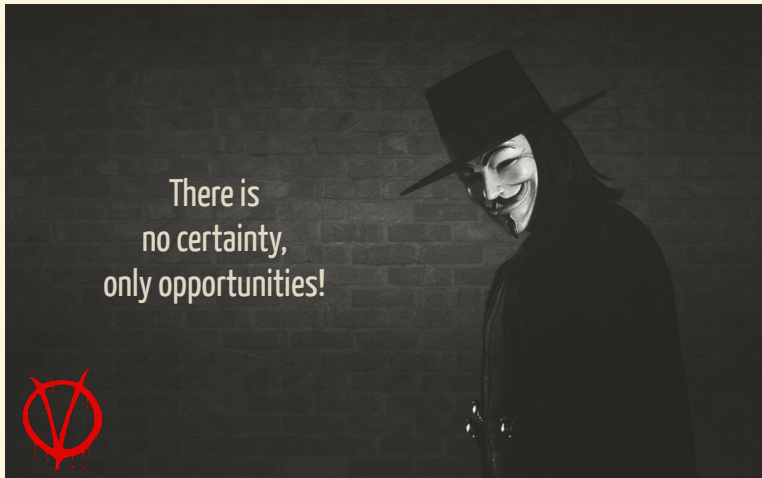
In academia

- Development is rather slow and in small teams



Summary and outlook

Most used Git commands



Break

Clean testing (IV)

Test driven development as discipline

It is a discipline

It is a discipline

- Do not bring it immediately into your daily work
- Learn it in small exercise projects:
 - Implement a `Stack` object
 - Implement a file parser
 - Write a 1D integration tool with few different algorithms
 - Simulate the 2D Ising model with a Monte Carlo algorithm
 - Implement a simple game (e.g. hangman, minesweeper)
 - [...]
- Convince yourself about it
- Get a feeling and choose “your way”

Finally

Get rid of the “there-is-no-time-to-write-tests” attitude!



The purist original approach

Test driven development main rules

Robert C. Martin



The purist original approach

Test driven development main rules

- 1 You're not allowed to write any production code until you have first written a test that fails because the production code doesn't exist.

Robert C. Martin



The purist original approach

Test driven development main rules

- 1 You're not allowed to write any production code until you have first written a test that fails because the production code doesn't exist.
- 2 You're not allowed to write more of a test than is sufficient to fail. And not compiling is failing.

Robert C. Martin



The purist original approach

Test driven development main rules

- 1 You're not allowed to write any production code until you have first written a test that fails because the production code doesn't exist.
- 2 You're not allowed to write more of a test than is sufficient to fail. And not compiling is failing.
- 3 You're not allowed to write any more production code than is sufficient to pass the currently failing test.

Robert C. Martin



The purist original approach

Test driven development main rules

- 1 You're not allowed to write any production code until you have first written a test that fails because the production code doesn't exist.
- 2 You're not allowed to write more of a test than is sufficient to fail. And not compiling is failing.
- 3 You're not allowed to write any more production code than is sufficient to pass the currently failing test.

Robert C. Martin

How would it be a world in which
everything always worked one minute ago?



Writing the factorial in TDD

Source file

```
// Empty file
```

Test file

```
#include <vir/test.h>  
#include "factorial.h"
```

```
TEST(Nothing){}
```

It passes



Writing the factorial in TDD

Source file

```
// Empty file
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}
```

It doesn't compile



Writing the factorial in TDD

Source file

```
void factorial(int n){}
```

Notice the function signature!

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
            std::logical_error)
{
    factorial(-17);
}
```

It fails



Writing the factorial in TDD

Source file

```
void factorial(int n)
{
    throw std::logical_error();
}
```

Still no need to return anything!

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
            std::logical_error)
{
    factorial(-17);
}
```

It passes



Writing the factorial in TDD

Source file

```
void factorial(int n)
{
    throw std::logical_error();
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
}
```

It fails



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else
        return 1;
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
}

It passes
```



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if (n < 0)
        throw std::logical_error();
    else
        return 1;
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
}
```

It fails



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else if(n==1)
        return 1;
    else
        return n * factorial(n-1);
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
}
```

It passes



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else if(n==1)
        return 1;
    else
        return n * factorial(n-1);
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
    COMPARE(1, factorial(0));
}
```

It fails



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else if(n<2)
        return 1;
    else
        return n * factorial(n-1);
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
    COMPARE(1, factorial(0));
}
```

It passes



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else if(n<2)
        return 1;
    else
        return n * factorial(n-1);
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
    COMPARE(1, factorial(0));
    COMPARE(479001600,
            factorial(12));
}
```

It still passes



Writing the factorial in TDD

Source file

```
int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else if(n<2)
        return 1;
    else
        return n * factorial(n-1);
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"
#include <limits>

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
    COMPARE(1, factorial(0));
    COMPARE(479001600,
             factorial(12));
    COMPARE(INT_MAX,
             factorial(13));
}
```

It fails



Writing the factorial in TDD

Source file

```
#include <limits>

int factorial(int n)
{
    if(n<0)
        throw std::logical_error();
    else if(n<2)
        return 1;
    else if(n>=13)
        return INT_MAX;
    else
        return n * factorial(n-1);
}
```

Test file

```
#include <vir/test.h>
#include "factorial.h"

TEST_CATCH(Negative_factorial,
           std::logical_error)
{
    factorial(-17);
}

TEST(FactorialTest) {
    COMPARE(1, factorial(1));
    COMPARE(120, factorial(5));
    COMPARE(1, factorial(0));
    COMPARE(479001600,
            factorial(12));
    COMPARE(INT_MAX,
            factorial(13));
}
```

It passes

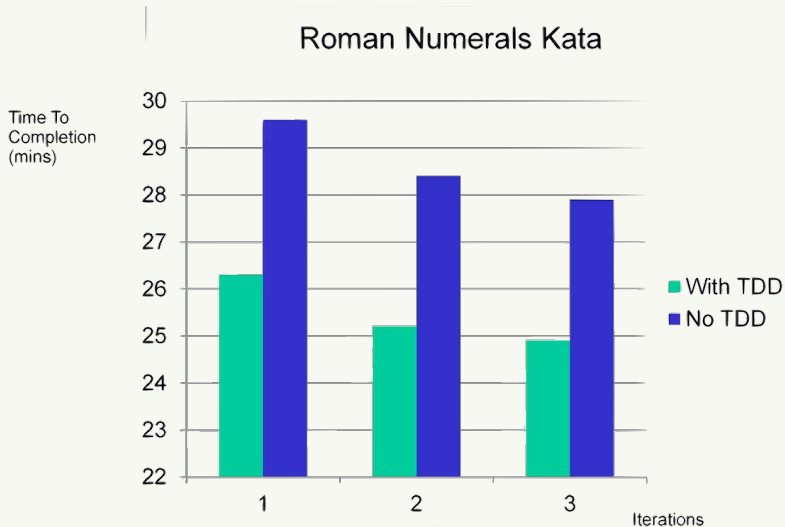


How strict should you be with yourself?



The more edits you do at once, the more dangerous it is!

Disciplines feel slow, they are not slow



<http://www.codemanship.co.uk/parlezuml/blog/?postid=1021>



The Red – Green – Refactor approach

What Kent Beck proposed

Red: Write a little test that doesn't work, and perhaps doesn't even compile at first.

Green: Make the test work quickly, committing whatever sins necessary in the process.

Refactor: Eliminate all of the duplication created in merely getting the test to work.

Ian Cooper

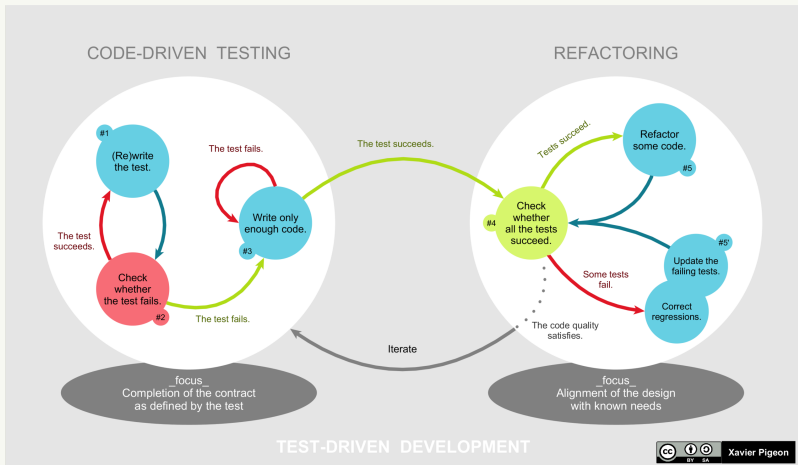
About testing the implementation

Sometimes it is useful to probe the implementation details because they are difficult to understand. You can do that, but **delete the tests afterwards!**

Ian Cooper



The Red – Green – Refactor approach



BDD as alternative

It is not **Bug** Driven Development!



Behaviour driven development in its original idea

- Dan North's  original idea stemmed from TDD
 - Test method names should be sentences
 - An expressive test name is helpful when a test fails
 - Determine the next most important behaviour
 - Requirements are behaviour, too
 - BDD provides a “ubiquitous language” for analysis



Behaviour driven development in its original idea

- Dan North's  original idea stemmed from TDD
- Use cases become stories and each test implements an acceptance criterion for the story

Example of use case as a story

Title: Customer withdraws cash

As a customer,
I want to withdraw cash from an ATM,
so that I don't have to wait in line at the bank.



Behaviour driven development in its original idea

- Dan North's  original idea stemmed from TDD
- Use cases become stories and each test implements an acceptance criterion for the story

Example of story acceptance criterion

Scenario: Account is overdrawn past the overdraft limit

Given the account is overdrawn

And the card is valid

When the customer requests cash

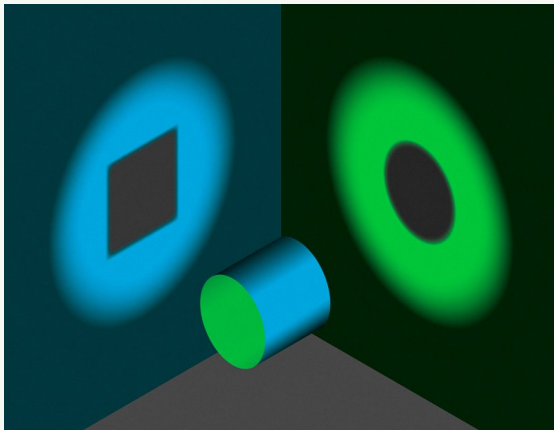
Then ensure a rejection message is displayed

And ensure cash is not dispensed

And ensure the card is returned



TDD and BDD



After all they are disciplines to deal with the same idea!

Take-home message

Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development
- 3 Tests are the only way to change code still ensuring its correctness



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development
- 3 Tests are the only way to change code still ensuring its correctness
- 4 Test the behaviour, not the implementation



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development
- 3 Tests are the only way to change code still ensuring its correctness
- 4 Test the behaviour, not the implementation
- 5 Depending on the size of the project, you can go one or another way



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development
- 3 Tests are the only way to change code still ensuring its correctness
- 4 Test the behaviour, not the implementation
- 5 Depending on the size of the project, you can go one or another way
- 6 If you do not want to strictly do TDD, still envision your tests and API usage before implementing any code



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development
- 3 Tests are the only way to change code still ensuring its correctness
- 4 Test the behaviour, not the implementation
- 5 Depending on the size of the project, you can go one or another way
- 6 If you do not want to strictly do TDD, still envision your tests and API usage before implementing any code
- 7 Now you are aware of how writing code should be done!



Wrapping up the advice

- 1 «I do not have time to write tests» **does not make any sense!**
- 2 Writing tests is an inseparable part of the code development
- 3 Tests are the only way to change code still ensuring its correctness
- 4 Test the behaviour, not the implementation
- 5 Depending on the size of the project, you can go one or another way
- 6 If you do not want to strictly do TDD, still envision your tests and API usage before implementing any code
- 7 Now you are aware of how writing code should be done!
- 8 Testing (and hence writing) code is doing science



And now?



And now?



If this is not your reaction right now, it will be soon!

TDD live demo

One of the best demo available on the web!

It is in Java, but do not worry, that's irrelevant!



Lunch break

Practice TDD (I)

Kata exercises: short tasks to practice TDD

Some from the  TDD manifesto

- 1 FizzBuzz
- 2 String calculator
- 3 Password input field validation
- 4 Search functionality
- 5 Point of sale kata
- 6 Banking kata

More kata exercises

The  TDD Buddy website offers some further challenges and few of them even propose a solution in `Python` or `C#`.



Your tasks: 90 minutes

- 0 You will work alone: Pick one programming language
- 1 Read the tasks and choose an appealing one
 - ↳ Start basic if you are new to TDD
- 2 Think about TDD rules
 - ↳ Decide how purist you want to be
- 3 Solve the problem using TDD
 - ↳ It will feel clumsy and slow...
- 4 Check where you stand after 30-45 minutes
- 5 Once you are done start over from scratch again
 - ↳ Now you can focus on the discipline more



Break

Practice TDD (II)

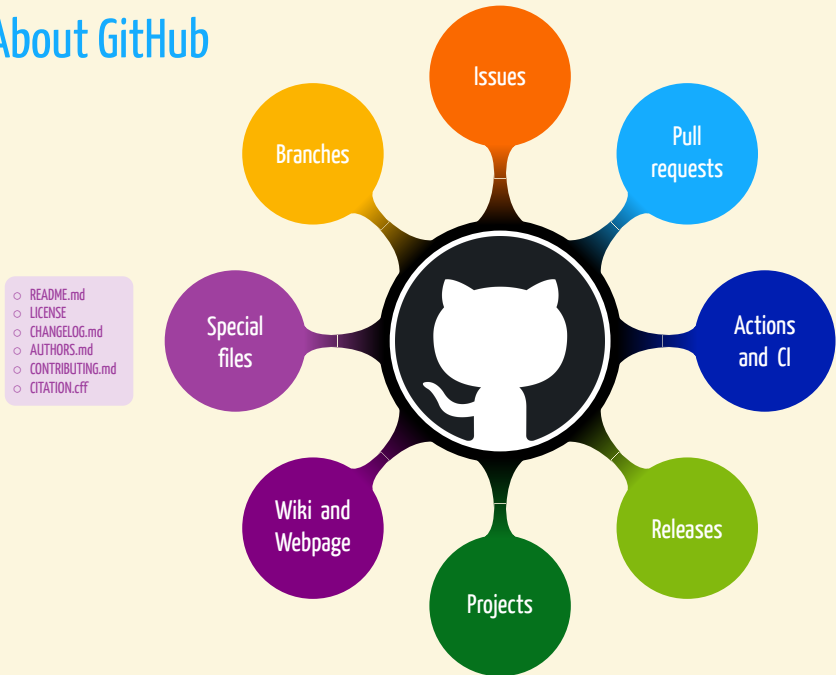
Your tasks: 45 minutes

- 6 Pair with another person { have (at least) a common programming language }
- 7 Pick up another exercise and solve it
 - ↳ One person write tests and the other writes code { Change roles at least once }



GitHub PR & co.

About GitHub

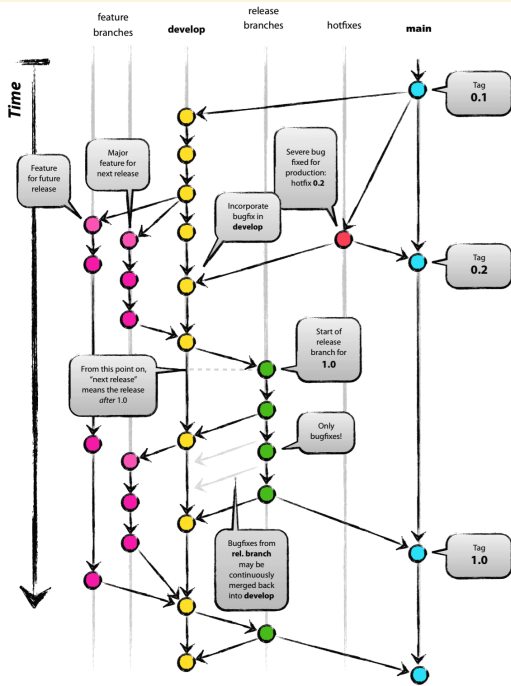


Branches

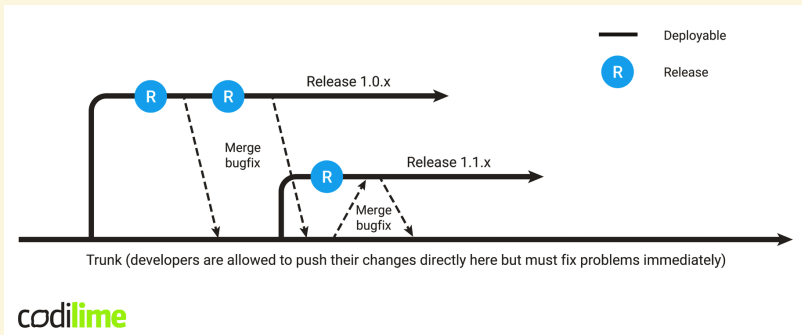
Branches



Git-flow branching model



Trunk-based development





Branches

When does trunk-based development work best?

- When you need to iterate quickly
- When you work mostly with senior developers

When does Git-flow work best?

- When you have a lot of junior developers
- When you have an established product
- When you run an open-source project

In academia

- Development is rather slow and in small teams



Issues



Issues

A way to organise your (team's) work

When opening the issue

- 1 Choose a descriptive title
- 2 Spend time on description!
- 3 Assign somebody whenever possible
- 4 Consistently use labels for issues properties
- 5 Consistently use types to describe the issue entity
- 6 Assign the issue to a project if reasonable
- 7 Add the issue to a milestone if it makes sense





Issues

This is how it might look like

 **Create new issue**

Add a title *

Add a description

WritePreview

HBI≡<>🔗≡≡≡📎🔄👍

Type your description here...

 Paste, drop, or click to add files

☐ Create more

Cancel

Create ⌘ ↵

Assignees ⚙️
No one - [Assign yourself](#)

Labels ⚙️
No labels

Type ⚙️
No type

Projects ⚙️
No projects

Milestone ⚙️
No milestone

① Remember, contributions to this repository should follow its [contributing guidelines](#).



Pull requests



Pull requests

A way to contribute to a repository

A clever merge procedure

A pull request is a way to prepare a merge of a Git branch into another one. Through a graphic interface, the changes can be reviewed and actions can be done.

Draft RenHirayama wants to merge 13 commits into `main` from `RenHirayama/add-max-p-option-dynflu`

Conversation 1 · Commits 13 · Checks 2 · Files changed 24 +236 -88

RenHirayama commented on Jan 29 · edited · ...

This imposes a maximum momentum in the selection of fluidizable particles, and refactors the code a bit. I chose the default $\max(|p|)=7$ GeV as that did not break any of the 20 vHLL runs. In any case this is already way faster than what's usually considered bulk.

@EggerJoscha this is just so you learn how to review a PR. See if the changes make some sense for you, and if the description of the key is understandable.

This closes #1346.

Also @yukarpenko has updated vHLL from float to double precision in the TransformPV function, which was the original problem, but I think this option still makes physical sense.

🔗 **RenHirayama** requested review from **zpauliny** and **EggerJoscha** 2 months ago

⚙️ **RenHirayama** marked this pull request as draft last month

Reviewers

- zpauliny**
- EggerJoscha**

At least 1 approving review is required to merge this pull request.

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone





Pull requests

A way to contribute to a repository

A clever merge procedure

A pull request is a way to prepare a merge of a Git branch into another one. Through a graphic interface, the changes can be reviewed and actions can be done.

- Always assign at least one reviewer
- A PR can be opened as a **Draft**
- Use automatic linking to issues in the description
- **Actions automatically start**
- The review is done in the “Files changed” tab
- The reviewer comments on code and can request changes





PR reviewing process

An example of typical workflow

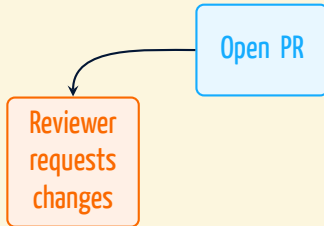
Open PR





PR reviewing process

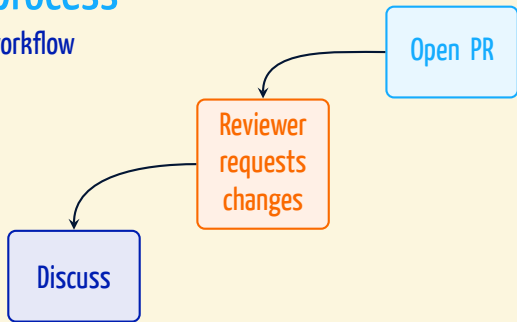
An example of typical workflow





PR reviewing process

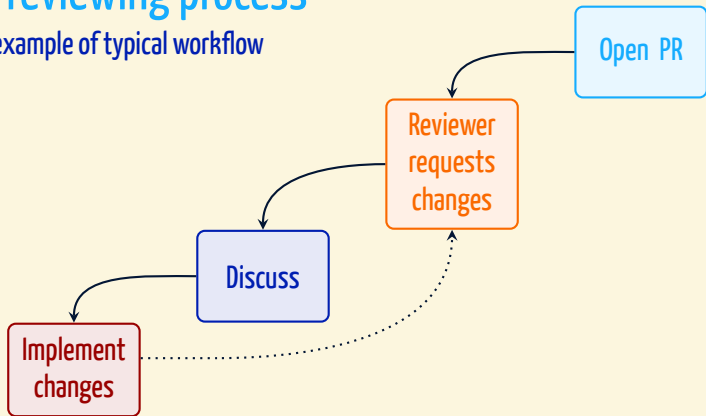
An example of typical workflow





PR reviewing process

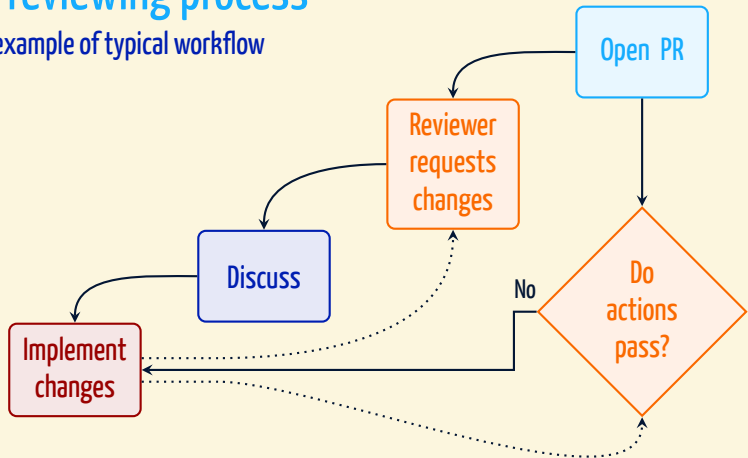
An example of typical workflow





PR reviewing process

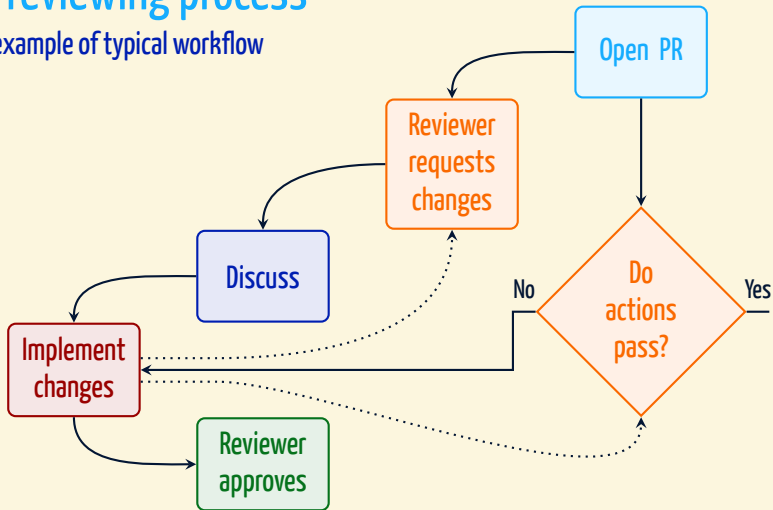
An example of typical workflow





PR reviewing process

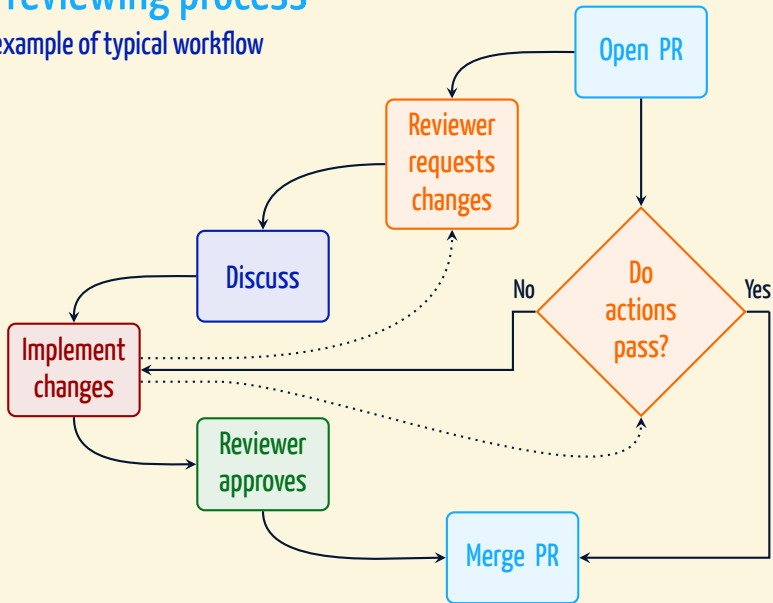
An example of typical workflow





PR reviewing process

An example of typical workflow



Actions and CI

Actions and continuous integration

A world plenty of opportunities!





Actions and continuous integration

A world plenty of opportunities!

List of all possible events that can trigger an action

branch_protection_rule	page_build
check_run	public
check_suite	pull_request
create	pull_request_comment
delete	pull_request_review
deployment	pull_request_target
deployment_status	push
discussion	registry_package
discussion_comment	release
fork	repository_dispatch
gollum	schedule
issue_comment	status
issues	watch
label	workflow_call
merge_group	workflow_dispatch
milestone	workflow_run

Find out more about  events that trigger a workflow !





Actions and continuous integration

Some typical examples

Actions use YAML files to configure what needs to be done!

1 Run tests for default PR activity

{ The workflow runs when a pull request is opened or reopened or when the head branch of the pull request is updated }

```
on:
  pull_request:
    branches:
      - main
      - release*
```

2 Run tests when merged code into some branches

```
on:
  push:
    branches: [main, develop, release*]
```





Actions and continuous integration

Some typical examples

3 Selecting GitHub virtual machines and run something

```
jobs:
  Run-on-Ubuntu:
    name: "Test on latest Ubuntu"
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: "Print a greeting"
        env:
          MY_VAR: "Hi there"
        run: |
          echo "${MY_VAR}!"
```

4 Using a self-hosted runner

```
runs-on: [self-hosted, linux, x64]
```

{ If you specify an array of strings or variables, your workflow will execute on any runner that matches all of the specified `runs-on` values. }





Actions and continuous integration

Standard GitHub-hosted runners for public repositories



GitHub Docs

Version: Free, Pro, & Team

▼

≡

GitHub Actions

/

Write workflows

/

Workflow syntax

Virtual Machine	Processor (CPU)	Memory (RAM)	Storage (SSD)	Architecture	Workflow label
Linux	4	16 GB	14 GB	x64	ubuntu-latest , ubuntu-24.04 , ubuntu-22.04 , ubuntu-20.04
Windows	4	16 GB	14 GB	x64	windows-latest , windows-2025 [Public preview], windows-2022 , windows-2019
Linux [Public preview]	4	16 GB	14 GB	arm64	ubuntu-24.04-arm , ubuntu-22.04-arm
macOS	4	14 GB	14 GB	Intel	macos-13
macOS	3 (M1)	7 GB	14 GB	arm64	macos-latest , macos-14 , macos-15 [Public preview]

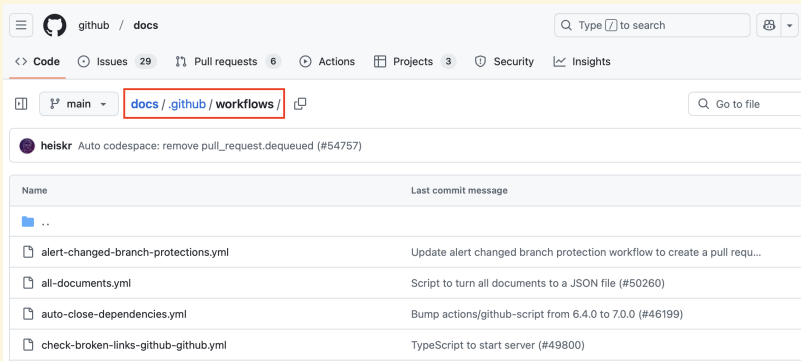




Creating your first workflow

Taken from  the official actions quick-start

1 Action files need to be placed in the `.github/workflows` folder



github / docs

Code Issues 29 Pull requests 6 Actions Projects 3 Security Insights

main docs / .github / workflows /

heiskr Auto codespace: remove pull_request.dequeued (#54757)

Name	Last commit message
..	
alert-changed-branch-protections.yml	Update alert changed branch protection workflow to create a pull requ...
all-documents.yml	Script to turn all documents to a JSON file (#50260)
auto-close-dependencies.yml	Bump actions/github-script from 6.4.0 to 7.0.0 (#46199)
check-broken-links-github-github.yml	TypeScript to start server (#49800)





Creating your first workflow

Taken from [the official actions quick-start](#)

2 Action files are written using [YAML workflows syntax](#)

YAML

```
name: GitHub Actions Demo
run-name: ${ github.actor }} is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${ github.event_name }} event."
      - run: echo "🖥️ This job is now running on a ${ runner.os }} server hosted by GitHub!"
      - run: echo "🔍 The name of your branch is ${ github.ref }} and your repository is ${ github.repository }}."
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${ github.repository }} repository has been cloned to the runner."
      - run: echo "🖨️ The workflow is now ready to test your code on the runner."
      - name: List files in the repository
        run: |
          ls ${ github.workspace }}
      - run: echo "🍏 This job's status is ${ job.status }}."
```

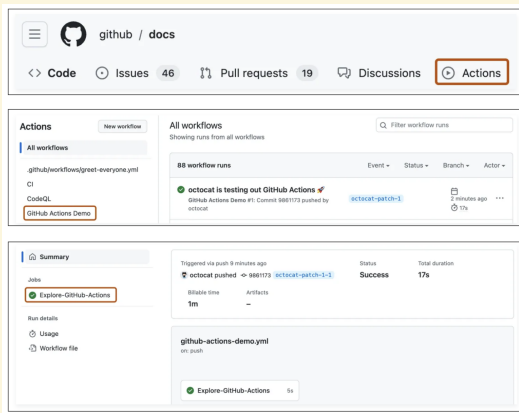




Creating your first workflow

Taken from  the official actions quick-start

3 Actions are triggered by the event you chose in the YAML file



The screenshot shows the GitHub Actions interface for the repository 'github / docs'. The 'Actions' tab is selected, showing a list of workflow runs. The 'octocat is testing out GitHub Actions' workflow is highlighted, showing it was triggered by a push event on the 'octocat-patch-1' branch. The 'Summary' section shows the job 'Explore-GitHub-Actions' with a status of 'Success' and a duration of 17s. The 'Run details' section shows the workflow file 'github-actions-demo.yml' and the job 'Explore-GitHub-Actions' with a status of 'Success' and a duration of 5s.

github / docs

<> Code Issues 46 Pull requests 19 Discussions Actions

Actions New workflow

All workflows Showing runs from all workflows

Filter workflow runs

88 workflow runs

Event	Status	Branch	Actor
octocat is testing out GitHub Actions	Success	octocat-patch-1	octocat

octocat is testing out GitHub Actions

GitHub Actions Demo #1: Commit 9061173 pushed by octocat

2 minutes ago

17s

Summary

Jobs

Explore-GitHub-Actions

Run details

Usage

Workflow file

Triggered via push 9 minutes ago

octocat pushed 9061173 octocat-patch-1

Status Success

Total duration 17s

Billable time 1m

Artifacts -

github-actions-demo.yml

on: push

Explore-GitHub-Actions 5s





Creating your first workflow

Taken from  the official actions quick-start

4 Actions log file is available on GitHub

Explore-GitHub-Actions
succeeded 13 minutes ago in 5s

Search logs

>  Set up job	1s
>  Run echo "🚀 The job was automatically triggered by a push event."	0s
>  Run echo "🔥 This job is now running on a Linux server hosted by GitHub!"	0s
>  Run echo "📌 The name of your branch is refs/heads/octocat-patch-1 and your repository is oct..."	0s
>  Check out repository code	1s
>  Run echo "💡 The octo-org/octo-repo repository has been cloned to the runner."	0s
>  Run echo "🖨️ The workflow is now ready to test your code on the runner."	0s
>  List files in the repository	0s
>  Run echo "🟢 This job's status is success."	0s
>  Post Check out repository code	0s
>  Complete job	0s



Releases



Releases

An interface to associate metadata to a Git tag

AxelKrypton / Bash-lecture

Q Type / to search

<> Code Issues Pull requests Actions Projects Security Insights Settings

Releases Tags

Draft a new release Find a release

3 weeks ago

AxelKrypton

v2025

1f43f88

Compare

Full 2025 lecture material Latest

At the end of each lecture and exercise session, the material has been revised and some typos have been fixed. All the lecture material is here attached in a unique PDF file (including exercises). Hyperlinks and bookmarks should work as in the files of the single days. A new file collecting all links from all slides to read material beyond the lecture has been added to the repository and is attached here again for convenience.

▼ Assets 4

Full_Bash_Lecture_2025.pdf	92.5 MB	3 weeks ago
Useful_links.md	10.6 KB	3 weeks ago
Source code (zip)		3 weeks ago
Source code (tar.gz)		3 weeks ago

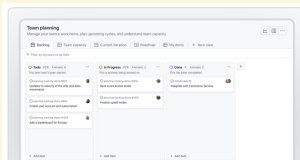


Projects



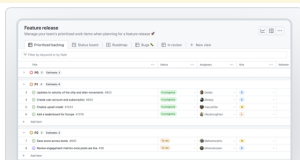
Projects

Functionality at the Organisation or User level



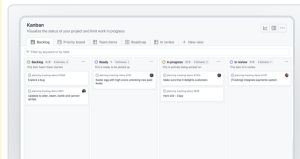
Team planning • GitHub

Manage your team's work items, plan upcoming cycles, and understand team capacity



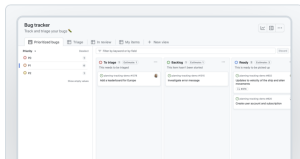
Feature release • GitHub

Manage your team's prioritized work items when planning for a feature release 🚀



Kanban • GitHub

Visualize the status of your project and limit work in progress



Bug tracker • GitHub

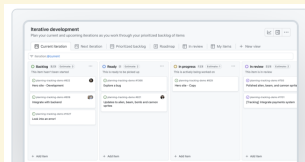
Track and triage your bugs 🐛





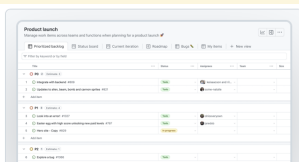
Projects

Functionality at the Organisation or User level



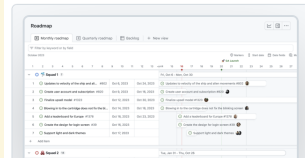
Iterative development • GitHub

Plan your current and upcoming iterations as you work through your prioritized backlog of items



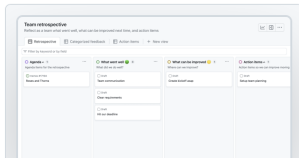
Product launch • GitHub

Manage work items across teams and functions when planning for a product launch



Roadmap • GitHub

Manage your team's long term plans as you plan out your roadmap



Team retrospective • GitHub

Reflect as a team what went well, what can be improved next time, and action items



Wiki and webpage



Wiki and webpage

Wiki pages

You can enable and create Wiki pages associated to every repository. This is a separate repository with a dedicated UI bound to your main codebase.

Home

Matt Jacobs edited this page on Jul 3, 2017 - [24 revisions](#)



HYSTRIX

DEFEND YOUR APP

- [What Is Hystrix?](#)
- [What Is Hystrix For?](#)
- [What Problem Does Hystrix Solve?](#)
- [What Design Principles Underlie Hystrix?](#)
- [How Does Hystrix Accomplish Its Goals?](#)

Pages **13**

- [Home](#)
- [Getting Started](#)
- [How It Works](#)
- [How To Use](#)
- [Operations](#)
- [Configuration](#)
- [Metrics](#)
- [Plugins](#)
- [End-to-End Examples](#)
- [Migration Guide](#)
- [FAQ - General](#)
- [FAQ - Operational](#)

Clone this wiki locally

<https://github.com/Netflix/Hystr> 

What Is Hystrix?



Wiki and webpage

GitHub pages

You can create a website associated to your organisation/user or to any repository!



Special files



Special files

README.md An overview of your codebase

LICENSE How is your material shipped to the public

AUTHORS.md A list of people that contributed

CONTRIBUTING.md How to become new developers

CHANGELOG.md User-relevant changes between releases

CITATION.cff How to cite your software

Take advantage of this!

Most files usually use  **Markdown formatting**



Sum up and closing

Time to reflect on the day

You have again an opportunity after dinner!

- ✓ Finish working on exercises in the room
- ✓ Ask further and/or broader questions
- ✓ Enjoy others' discussion

Dinner	Dinner	Dinner	Dinner	
	Informal discussion Office hour	Informal discussion Office hour	Informal discussion Office hour	

It is not mandatory, but you can use it as you wish.



Time to reflect on the day

1 Take few minutes to look back at the work done:

- How was the digression on Git usage?
- What's your feeling about TDD?
- How was the practical session about TDD?
- Are there unanswered questions?
- Are you sceptical? If so, about what?

2 Let's discuss your experience/work!

